LAPORAN PRAKTIKUM
STRUKTUR DATA

Modul 6
DOUBLY LINKED LIST (BAGIAN PERTAMA)



Disusun Oleh :
Dina Aulia Rizky (101902500011)

Dosen:
Diah Septiani, S.Kom., M.C.S.(AI)

PROGRAM STUDI S-1 SAINS DATA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM
2026

Doublylist.h

```
/* file : Doublylist.h */
/* ADT Doubly Linked List dengan representasi fisik pointer */
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <iostream>
#include <string>
using namespace std;

#define Nil NULL

struct kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address First;
    address Last;
};

/* ===== Soal 1 ===== */
void createList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);
```

```
/* ===== Soal 2 ===== */
address findElm(List L, infotype x);

/* ===== Soal 3 ===== */
void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);

#endif
```

Doublylist.cpp

```
/* file : Doublylist.cpp */
/* Implementasi ADT Doubly Linked List */
#include "Doublylist.h"

void createList(List &L) {
    L.First = Nil;
    L.Last = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    P->prev = Nil;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

/* Ditampilkan dari Last ke First sesuai Gambar 6-23 */
void printInfo(List L) {
    address P = L.Last;
    while (P != Nil) {
```

```
        cout << "no polisi : " << P->info.nopol << endl;
        cout << "warna     : " << P->info.warna << endl;
        cout << "tahun     : " << P->info.thnBuat << endl;
        P = P->prev;
    }
}

void insertLast(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
        L.Last = P;
    } else {
        P->prev = L.Last;
        L.Last->next = P;
        L.Last = P;
    }
}

/* Pencarian berdasarkan nopol */
address findElm(List L, infotype x) {
    address P = L.First;
    while (P != Nil) {
        if (P->info.nopol == x.nopol) {
            return P;
        }
        P = P->next;
    }
    return Nil;
}

void deleteFirst(List &L, address &P) {
    P = L.First;
    if (P == Nil) return;
    if (L.First == L.Last) {
        L.First = Nil;
        L.Last = Nil;
    } else {
        L.First = P->next;
        L.First->prev = Nil;
        P->next = Nil;
```

```
    }
}

void deleteLast(List &L, address &P) {
    P = L.Last;
    if (P == Nil) return;
    if (L.First == L.Last) {
        L.First = Nil;
        L.Last = Nil;
    } else {
        L.Last = P->prev;
        L.Last->next = Nil;
        P->prev = Nil;
    }
}

/* P = elemen setelah Prec, P dilepas dari list */
void deleteAfter(address Prec, address &P) {
    P = Prec->next;
    if (P == Nil) return;
    Prec->next = P->next;
    if (P->next != Nil) {
        P->next->prev = Prec;
    }
    P->next = Nil;
    P->prev = Nil;
}
```

main.cpp

```
/* file : main.cpp */
#include "Doublylist.h"

/* Tampilan list setelah hapus (label sesuai Gambar 6-25) */
void printHasilHapus(List L) {
    address P = L.Last;
    while (P != Nil) {
        cout << "Nomor Polisi : " << P->info.nopol << endl;
```

```
        cout << "Warna        : " << P->info.warna << endl;
        cout << "Tahun        : " << P->info.thnBuat << endl;
        P = P->prev;
    }
}

int main() {
    List L;
    infotype x;
    address P;

    createList(L);

    /* ===== Soal 1 : input 4 data ===== */
    for (int i = 0; i < 4; i++) {
        cout << "masukkan nomor polisi: ";
        cin >> x.nopol;
        cout << "masukkan warna kendaraan: ";
        cin >> x.warna;
        cout << "masukkan tahun kendaraan: ";
        cin >> x.thnBuat;

        if (findElm(L, x) != Nil) {
            cout << "nomor polisi sudah terdaftar" << endl;
        } else {
            insertLast(L, alokasi(x));
        }
        cout << endl;
    }

    cout << "DATA LIST 1" << endl << endl;
    printInfo(L);
    cout << endl;

    /* ===== Soal 2 : cari nopol ===== */
    cout << "Masukkan Nomor Polisi yang dicari  : ";
    cin >> x.nopol;
    cout << endl;
    P = findElm(L, x);
    if (P != Nil) {
```

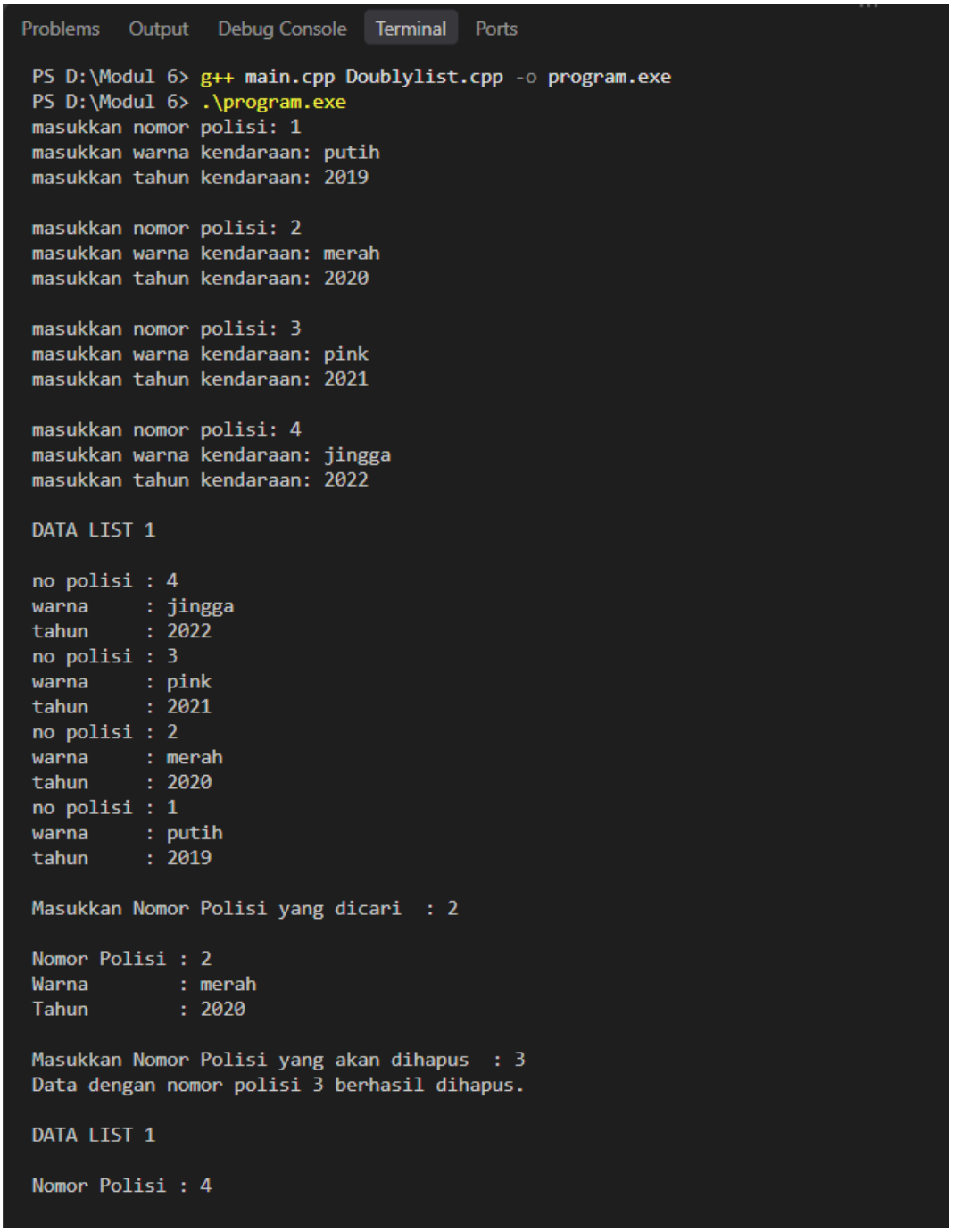
```
        cout << "Nomor Polisi : " << P->info.nopol << endl;
        cout << "Warna        : " << P->info.warna << endl;
        cout << "Tahun        : " << P->info.thnBuat << endl;
    } else {
        cout << "Data dengan nomor polisi " << x.nopol << "
tidak ditemukan." << endl;
    }
    cout << endl;

    /* ===== Soal 3 : hapus nopol ===== */
    cout << "Masukkan Nomor Polisi yang akan dihapus  : ";
    cin >> x.nopol;
    P = findElm(L, x);
    if (P == Nil) {
        cout << "Data dengan nomor polisi " << x.nopol << "
tidak ditemukan." << endl;
    } else {
        address D;
        if (P == L.First) {
            deleteFirst(L, D);
        } else if (P == L.Last) {
            deleteLast(L, D);
        } else {
            deleteAfter(P->prev, D);
        }
        dealokasi(D);
        cout << "Data dengan nomor polisi " << x.nopol << "
berhasil dihapus." << endl;
    }

    cout << endl << "DATA LIST 1" << endl << endl;
    printHasilHapus(L);

    /* bersihkan memori */
    while (L.First != Nil) {
        deleteFirst(L, P);
        dealokasi(P);
    }
    return 0;
}
```

Output



Kesimpulan

Berdasarkan praktikum yang telah dilakukan, ADT Doubly Linked List berhasil diimplementasikan dengan representasi pointer pada file Doublylist.h, Doublylist.cpp, dan main.cpp. Setiap elemen (ElmList) menyimpan data

kendaraan (nomor polisi, warna, dan tahun buat) serta dua pointer, yaitu next yang menunjuk ke elemen berikutnya dan prev yang menunjuk ke elemen sebelumnya. Dengan dua pointer ini, list bisa ditelusuri dua arah. Pada program ini, data ditampilkan dari Last ke First, sehingga data yang terakhir dimasukkan muncul lebih dulu.